

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 16: AVALIAÇÃO DE DESEMPENHO COM SCATTER, BROADCAST E GATHER

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 18/09/2026.

1. EXPERIMENTO

Implemente um programa MPI que calcule o produto $y=A \cdot x$, onde A é uma matriz M×N e x é um vetor de tamanho N. Divida a matriz A por linhas entre os processos com MPI_Scatter, e distribua o vetor x inteiro com MPI_Bcast. Cada processo deve calcular os elementos de y correspondentes às suas linhas e enviá-los de volta ao processo 0 com MPI_Gather. Compare os tempos com diferentes tamanhos de matriz e número de processos.

2. CONCEITOS RELEVANTES

- **MPI_Scatter:** o processo raiz divide um vetor ou matriz em partes e envia uma parte para cada processo. No programa, divide as linhas de A: cada processo recebe seu bloco local de linhas.
- **MPI_Bcast**: o processo raiz envia a mesma informação para todos os processos. No programa, o vetor inteiro x é copiado para todos, pois cada processo precisa dele para calcular suas linhas de y.
- **MPI_Gather**: cada processo envia sua parte do resultado ao processo raiz, que junta tudo na ordem correta. No programa, cada processo envia seus elementos locais de y, e o processo 0 forma o vetor y completo.

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador AMD Ryzen 7 5700X (8 núcleos físicos e 16 threads). O sistema dispõe também de 32 GB de memória RAM e um SSD de 1TB.

4. CÓDIGO

O programa calcula o produto matriz-vetor “y = A × x” usando MPI. O processo 0 cria a matriz “A” e o vetor “x”. Em seguida, “MPI_Scatter” divide as linhas da matriz igualmente entre os processos.

O vetor “x” é necessário para todos os processos, então o processo 0 o distribui com “MPI_Bcast”. Cada processo calcula os elementos de “y” correspondentes às linhas de matriz que recebeu, multiplicando cada linha pelo vetor.

Ao final, “MPI_Gather” reúne os resultados parciais no processo 0, formando o vetor “y” completo. O tempo é iniciado antes das operações de comunicação e cálculo, e o processo 0 o encerra depois do “Gather”, registrando o tempo total da operação paralela.

Figura 1: Código utilizado

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static void preencherDados(double *matriz, double *vetor, int linhas, int colunas) {
    for (int i = 0; i < linhas * colunas; i++) {
        matriz[i] = (double)((i % 10) + 1);
    }

    for (int i = 0; i < colunas; i++) {
        vetor[i] = 1.0;
    }
}
```

```c
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int processo, quantidadeProcessos;
    MPI_Comm_rank(MPI_COMM_WORLD, &processo);
    MPI_Comm_size(MPI_COMM_WORLD, &quantidadeProcessos);

    int linhas = 1000;
    int colunas = 1000;

    if (argc == 3) {
        linhas = atoi(argv[1]);
        colunas = atoi(argv[2]);
    }

    // Verifica se o número de linhas é divisível pelo número de processos
    int linhasLocais = linhas / quantidadeProcessos;
    // cria matriz e vetor local para cada processo
    double *matrizM = NULL;
    double *vetorN = malloc((size_t)colunas * sizeof(double));
    // cria matriz local e vetor local para cada processo
    double *matrizLocal = malloc((size_t)linhasLocais * colunas * sizeof(double));
    double *resultadoLocal = malloc((size_t)linhasLocais * sizeof(double));
    double *resultado = NULL;

    if (processo == 0) {
        matrizM = malloc((size_t)linhas * colunas * sizeof(double));
        resultado = malloc((size_t)linhas * sizeof(double));
        preencherDados(matrizM, vetorN, linhas, colunas);
    }

    MPI_Barrier(MPI_COMM_WORLD);

    double inicio = MPI_Wtime();

    MPI_Scatter(
        matrizM, // linhas * colunas elementos da matriz
        linhasLocais * colunas, // cada processo recebe linhasLocais * colunas elementos
        MPI_DOUBLE, // tipo de dado
```

```
    matrizLocal, // buffer de recepção
    linhasLocais * colunas, // quantidade de elementos recebidos
    MPI_DOUBLE, // tipo de dado
    0, // processo raiz
    MPI_COMM_WORLD // comunicador
);
MPI_Bcast(
    vetorN, // vetor de tamanho
    colunas, // quantidade de elementos
    MPI_DOUBLE, // tipo de dado
    0, // processo raiz
    MPI_COMM_WORLD // comunicador
);

// Cada processo realiza a multiplicação da sua parte da matriz pelo vetor
for (int i = 0; i < linhasLocais; i++) {
    // Inicializa o resultado local para a linha i
    resultadoLocal[i] = 0.0;
    // Multiplica a linha i da matriz local pelo vetor
    for (int j = 0; j < colunas; j++) {
        // resultadoLocal[i] += matrizLocal[i][j] * vetorN[j];
        resultadoLocal[i] += matrizLocal[i * colunas + j] * vetorN[j];
    }
}

MPI_Gather(
    resultadoLocal,
    linhasLocais,
    MPI_DOUBLE,
    resultado,
    linhasLocais,
    MPI_DOUBLE,
    0,
    MPI_COMM_WORLD
);

if (processo == 0) {
    double tempo = MPI_Wtime() - inicio;
    printf("M=%d N=%d processos=%d tempo_s=%.6f y0=%.1f\n",
```

```
            linhas, colunas, quantidadeProcessos, tempo, resultado[0]);
    }

    free(matrizM);
    free(vetorN);
    free(matrizLocal);
    free(resultadoLocal);
    free(resultado);

    MPI_Finalize();

    return 0;
}
```

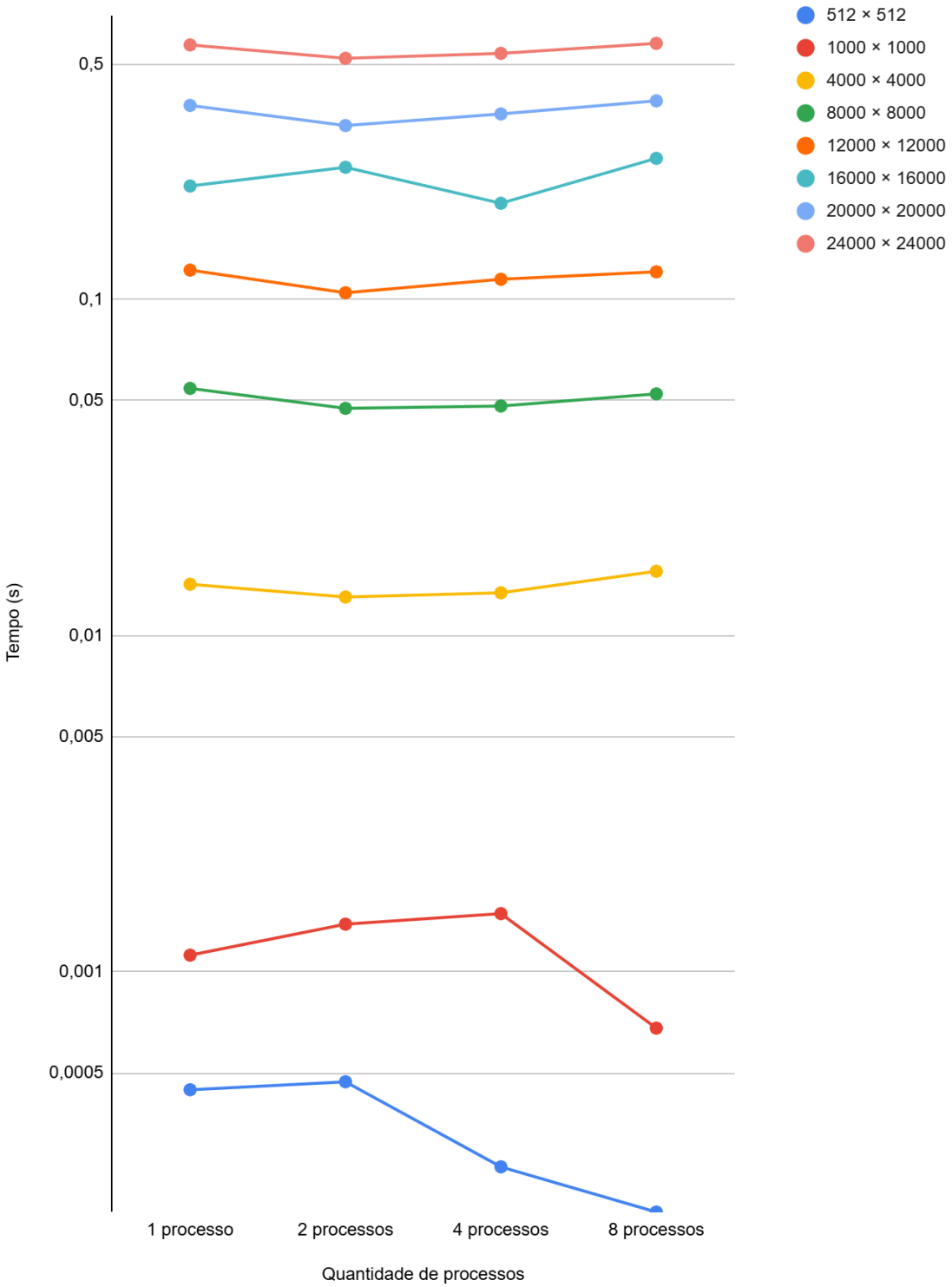
5. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho do experimento.

Tabela 1: Mediana variando número de processos e tamanho da matriz

Matriz	1 processo	2 processos	4 processos	8 processos
512 × 512	0,000446	0,000471	0,000263	0,000193
1000 × 1000	0,001121	0,001386	0,00149	0,00068
4000 × 4000	0,014214	0,013023	0,013406	0,015538
8000 × 8000	0,05436	0,047417	0,048169	0,052347
12000 × 12000	0,122185	0,104622	0,114765	0,120764
16000 × 16000	0,217347	0,247048	0,193111	0,262676
20000 × 20000	0,377424	0,328802	0,356001	0,38947
24000 × 24000	0,571196	0,521082	0,538986	0,577285

Figura 2: Mediana variando número de processos e tamanho da matriz



Os resultados mostram que aumentar o número de processos não produz ganho proporcional. Em geral, dois processos apresentaram os melhores tempos nas matrizes maiores, enquanto quatro e oito processos tiveram ganhos menores ou até piora.

Isso ocorre porque o tempo medido não inclui apenas a multiplicação matriz-vetor. Ele inclui as comunicações coletivas do MPI_Scatter (que distribui partes da matriz), do MPI_Bcast (que envia o vetor inteiro para todos) e do MPI_Gather (que reúne o vetor resultado).

Quando novos processos são adicionados, cada processo calcula menos linhas da matriz, reduzindo o trabalho local. Porém, a matriz ainda precisa ser distribuída, o vetor precisa alcançar todos os processos e os resultados precisam retornar ao processo 0. Esses custos de comunicação não diminuem na mesma proporção do cálculo e podem crescer com mais processos.

Além disso, o produto matriz-vetor acessa muitos dados na memória. Cada processo lê sua parte da matriz e o vetor completo. Com muitos processos no mesmo computador, todos disputam cache e largura de banda da memória. Depois de certo ponto, o processador deixa de ser o gargalo e a movimentação de dados passa a limitar o desempenho.

Para matrizes pequenas, como 512 × 512 e 1000 × 1000, o cálculo é muito rápido. Nesse caso, diferenças de microssegundos podem depender de cache, escalonamento do sistema operacional e variações normais de execução, então os resultados com mais processos não são tão representativos.

Para matrizes maiores, há trabalho suficiente para justificar algum paralelismo, mas o ganho observado ainda foi modesto. Isso é coerente com a arquitetura do programa, já que o processo 0 é responsável por distribuir toda a matriz e reunir todo o resultado, tornando-se um ponto central de comunicação. Assim, dois processos normalmente equilibraram melhor a redução do cálculo com o custo extra de comunicação. Quatro e oito processos aumentaram a concorrência e os custos de comunicação e memória sem reduzir o tempo total na mesma proporção.